[인공지능]

Learning in Artificial Neural Networks

Eunsong Kang
eskang@kangwon.ac.kr

2026. 11. 9.

Concept of Learning in Neural Networks

- **Learning** refers to the process of automatically acquiring optimal weight parameters from training data.
- A **loss function** provides a quantitative measure of how well the neural network performs.
- The goal of learning is to find the weight/bias parameters that minimize the loss value.
- To achieve this, neural networks employ the gradient-based optimization method (**Gradient Descent**), which uses the slope (derivative) of the loss function to update parameters.

→ Learning = Minimizing the loss function by adjusting weights in the direction of the steepest descent.

Loss Functions

- **Sum of squares for error**

$$E = \frac{1}{2}\sum_k (y_k - t_k)^2$$

  - k: number of dimension
  - y_k: output of the neural networks (predicted probability)
  - t_k: target label

- **Cross entropy error**

$$E = -\sum_k t_k \log y_k$$

  - t_k is one-hot encoded: 1 for the correct class, 0 for others.
  - When the prediction is correct, the term becomes $\log y_k$ for the corresponding class.
  - The negative sign ensures the loss is positive since $\log$ values are negative for $y_k < 1$.

KNU KANGWON NATIONAL UNIVERSITY

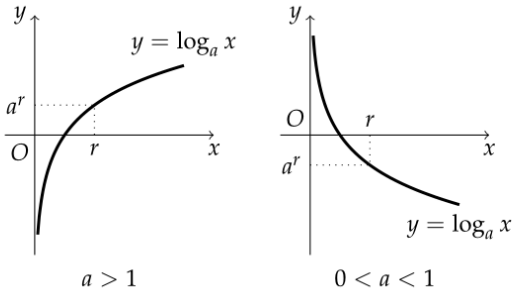
- One-hot encoding

Color
Red
Blue
Green

One-Hot Encoding

Color_Red	Color_Blue	Color_Green
1	0	0
0	1	0
0	0	1

- Log function



Mini-batch Learning in Neural Networks

- The total loss should be computed using all N training samples:

$$E = -\frac{1}{N} \sum_{n} \sum_{k} t_{nk} \log y_{nk}$$

  where:

  - t_{nk}: target value of the k-th output for the n-th data sample.
  - y_{nk}: predicted output for the same index.
  - N: total number of training samples.

- However, when N is very large (as in big data), computing the loss over all samples is computationally expensive.

- To overcome this, we use **mini-batch learning**.

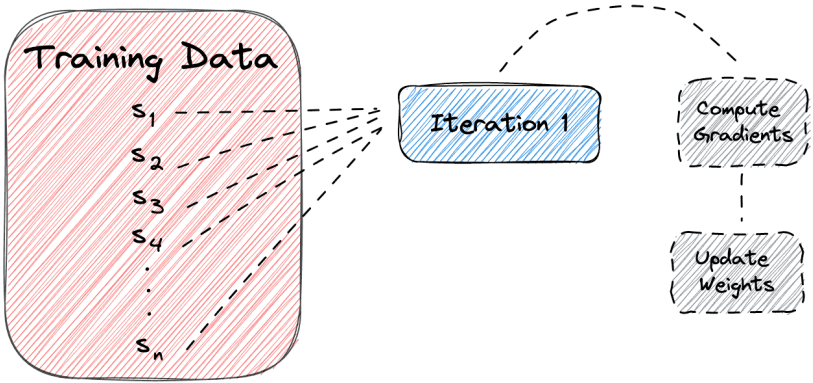
KNU KANGWON NATIONAL UNIVERSITY

Comparison: Batch vs. Mini-batch Learning

- **Batch learning**
  - The entire training dataset (all N samples) is treated as one batch.
  - The model computes the loss and updates the parameters once per epoch using all data.
  - This provides an accurate estimate of the gradient but requires large memory and computation time.

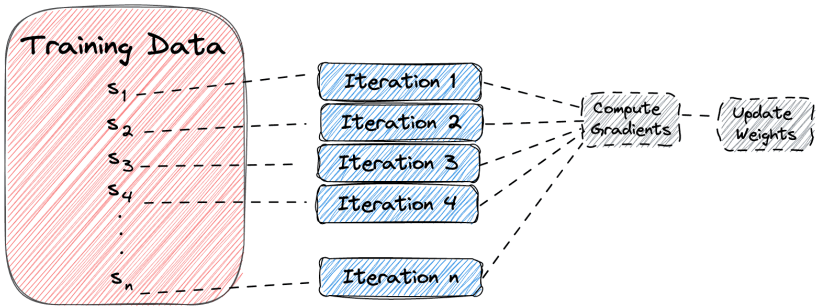
- **Mini-batch learning**
  - The dataset is divided into several smaller batches (e.g., 32–256 samples each).
  - Each mini-batch is used to compute an approximate gradient and update the model parameters.
  - This allows faster convergence and more stable updates compared to using a single sample or the full dataset.

- Batch gradient descent
  - In batch gradient descent, one epoch corresponds to a single iteration.



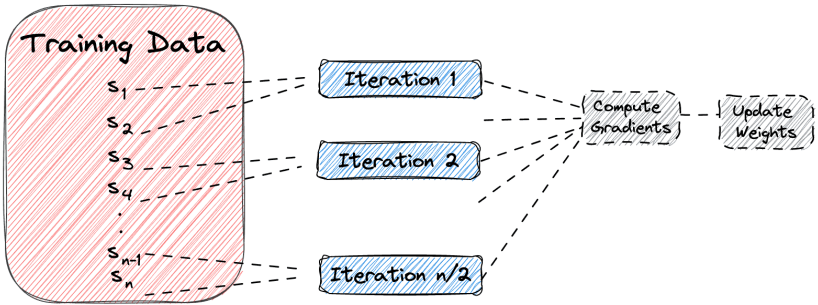
Source: Baeldung

- Stochastic gradient descent
  - In stochastic gradient descent, one epoch corresponds to n iterations where n is the number of training samples.



Source: Baeldung

- Mini-batch gradient descent
  - In mini-batch gradient descent, one epoch corresponds to $\frac{n}{b}$ iterations where b is the size of the mini-batch.



Source: Baeldung

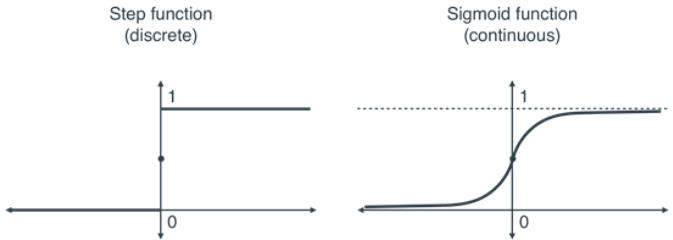
Why We Define a Loss Function?

- The learning process involves:
  - Calculating the derivative (gradient) of the loss function with respect to each parameter.
  - Updating the parameter values in the direction that decreases the loss.
- Suppose a network focuses on one particular parameter:
  - The gradient represents how the loss changes when this parameter slightly varies.
  - If the gradient is **negative**, increasing the parameter reduces the loss.
  - If the gradient is **positive**, decreasing the parameter reduces the loss.
  - If the gradient is **zero**, the loss does not change — parameter updates stop.
- Therefore, a differentiable loss function is essential for gradient-based learning.
- Metrics like accuracy cannot be used as loss functions because their gradients are zero almost everywhere, meaning the parameters cannot be updated.

Step Function vs. Sigmoid Function

- A step function has a gradient of zero almost everywhere, which prevents learning because parameter updates cannot occur.
- The sigmoid function has a non-zero and continuously varying gradient.

→ This smooth gradient allows the neural network to learn effectively by adjusting its parameters in the right direction.



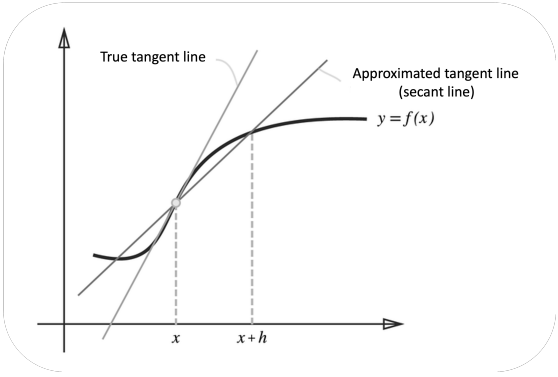
[Review] Numerical Differentiation

- **Analytical differentiation** gives an exact, formula-based result.
- **Numerical differentiation** provides an approximate value of the derivative by using small finite changes in the input.

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

  - $f(x+h) - f(x)$: Change in the function's value
  - $\frac{f(x+h)-f(x)}{h}$: Average rate of change
  - $\lim_{h \to 0}$: As h becomes infinitesimally small, the secant line approaches the tangent line, giving the instantaneous rate of change.

→ $\frac{df(x)}{dx}$ gives the slope of the tangent line at point x, showing how much and in what direction $f(x)$ is changing.



- Improvement 1: rounding error
  - h: A value of approximately 10^{-4} is generally regarded as a good choice.
- Improvement 2: central difference

$$\frac{df(x)}{dx} \approx \frac{f(x+h) - f(x-h)}{2h}$$

[Review] Partial Derivatives

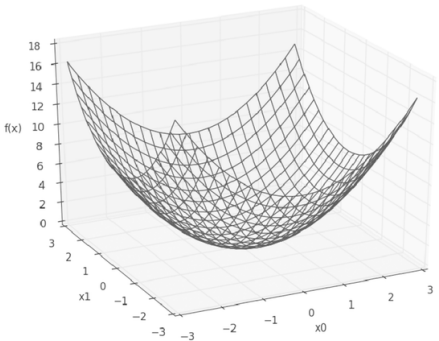
- In multivariable functions, the output depends on two or more variables.
- A **partial derivative** measures how the function changes with respect to one variable, while keeping the others constant.
- Partial derivatives isolate the effect of one variable at a time.

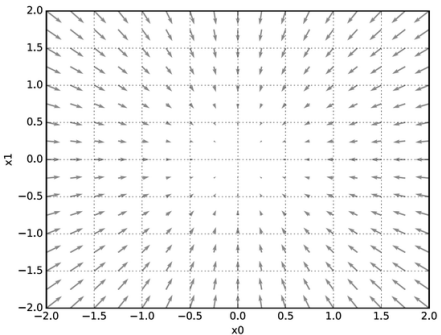
$$f(x_0, x_1) = x_0^2 + x_1^2 \Rightarrow \frac{\partial f}{\partial x_0} = 2x_0, \quad \frac{\partial f}{\partial x_1} = 2x_1$$

- Partial derivatives are used to update each weight parameter independently during optimization.

→ The collection of partial derivatives with respect to all variables is called the **gradient**.

$$f(x_0, x_1) = x_0^2 + x_1^2$$





KNU
KANGWON NATIONAL UNIVERSITY

Gradient Descent Method

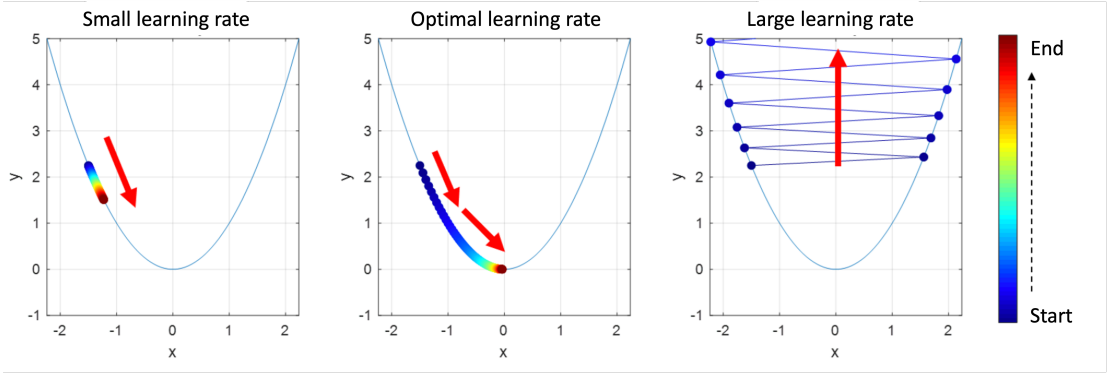
- The method used to find the minimum loss value by following the gradient is called **gradient descent**.
- The basic idea:
  1. Compute the gradient of the loss function at the current position.
  2. Move a small step in the opposite direction of the gradient (the direction of steepest decrease).
  3. Repeat this process until convergence.
- Mathematically, for parameters x_0 and x_1:

$$\begin{cases} x_0 = x_0 - \eta \frac{\partial f}{\partial x_0} \\ x_1 = x_1 - \eta \frac{\partial f}{\partial x_1} \end{cases}$$

where η is the **learning rate**, a constant that determines the step size of each update.

- Too large $\eta \rightarrow$ may overshoot the minimum.
- Too small $\eta \rightarrow$ convergence becomes very slow.

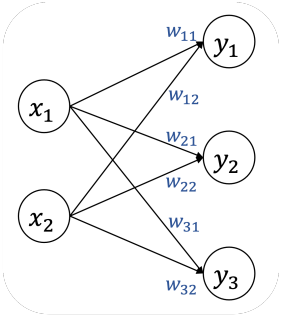
KNU KANGWON NATIONAL UNIVERSITY


Small learning rate
Optimal learning rate
Large learning rate
End
Start
x
y

Gradients in Neural Networks

- In neural network training, we must compute the gradient of the loss function with respect to each parameter (weight).

→ This gradient indicates how much each weight contributes to the overall loss and in which direction it should be updated.



$$\mathbf{W} = \begin{pmatrix} w_{11} & w_{21} & w_{31} \\ w_{12} & w_{22} & w_{32} \end{pmatrix}$$

$$\frac{\partial L}{\partial \mathbf{W}} = \begin{pmatrix} \frac{\partial L}{\partial w_{11}} & \frac{\partial L}{\partial w_{21}} & \frac{\partial L}{\partial w_{31}} \\ \frac{\partial L}{\partial w_{12}} & \frac{\partial L}{\partial w_{22}} & \frac{\partial L}{\partial w_{32}} \end{pmatrix}$$

Implementing the Learning Algorithm

- Neural network training typically consists of the following steps:

1. Mini-batch selection
   - Randomly select a small subset of the training data, called a **mini-batch**.
   - The goal is to minimize the loss function value computed from this mini-batch.
2. Gradient computation
   - Compute the gradient of the loss function with respect to each parameter using the mini-batch.
   - The gradient indicates the direction in which the loss increases most rapidly.
3. Parameter update
   - Update each parameter slightly in the direction opposite to the gradient.

   $$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

   where η is the learning rate.
4. Iteration
   - Repeat Steps 1–3 until convergence or for a fixed number of epochs.

```python
# coding: utf-8
import sys, os
sys.path.append(os.pardir)  # 부모 디렉터리의 파일을 가져올 수 있도록 설정
import numpy as np
import matplotlib.pyplot as plt
from dataset.mnist import load_mnist
from two_layer_net import TwoLayerNet

# 데이터 읽기
(x_train, t_train), (x_test, t_test) = load_mnist(normalize=True, one_hot_label=True)

network = TwoLayerNet(input_size=784, hidden_size=50, output_size=10)

# 하이퍼파라미터
iters_num = 10000  # 반복 횟수를 적절히 설정한다.
train_size = x_train.shape[0]
batch_size = 100   # 미니배치 크기
learning_rate = 0.1

train_loss_list = []
train_acc_list = []
test_acc_list = []

# 1에폭당 반복 수
iter_per_epoch = max(train_size / batch_size, 1)
```

```
27    for i in range(iters_num):
28        # 미니배치 획득
29        batch_mask = np.random.choice(train_size, batch_size)
30        x_batch = x_train[batch_mask]
31        t_batch = t_train[batch_mask]
32
33        # 기울기 계산
34        #grad = network.numerical_gradient(x_batch, t_batch)
35        grad = network.gradient(x_batch, t_batch)
36
37        # 매개변수 갱신
38        for key in ('W1', 'b1', 'W2', 'b2'):
39            network.params[key] -= learning_rate * grad[key]
40
41        # 학습 경과 기록
42        loss = network.loss(x_batch, t_batch)
43        train_loss_list.append(loss)
44
45        # 1에폭당 정확도 계산
46        if i % iter_per_epoch == 0:
47            train_acc = network.accuracy(x_train, t_train)
48            test_acc = network.accuracy(x_test, t_test)
49            train_acc_list.append(train_acc)
50            test_acc_list.append(test_acc)
51            print("train acc, test acc | " + str(train_acc) + ", " + str(test_acc))
```

KNU KANGWON NATIONAL UNIVERSITY

```python
# coding: utf-8
import sys, os
sys.path.append(os.pardir)  # 부모 디렉터리의 파일을 가져올 수 있도록 설정
from common.functions import *
from common.gradient import numerical_gradient


class TwoLayerNet:

    def __init__(self, input_size, hidden_size, output_size, weight_init_std=0.01):
        # 가중치 초기화
        self.params = {}
        self.params['W1'] = weight_init_std * np.random.randn(input_size, hidden_size)
        self.params['b1'] = np.zeros(hidden_size)
        self.params['W2'] = weight_init_std * np.random.randn(hidden_size, output_size)
        self.params['b2'] = np.zeros(output_size)

    def predict(self, x):
        W1, W2 = self.params['W1'], self.params['W2']
        b1, b2 = self.params['b1'], self.params['b2']

        a1 = np.dot(x, W1) + b1
        z1 = sigmoid(a1)
        a2 = np.dot(z1, W2) + b2
        y = softmax(a2)

        return y

    # x : 입력 데이터, t : 정답 레이블
    def loss(self, x, t):
        y = self.predict(x)

        return cross_entropy_error(y, t)

```

```python
    def accuracy(self, x, t):
        y = self.predict(x)
        y = np.argmax(y, axis=1)
        t = np.argmax(t, axis=1)

        accuracy = np.sum(y == t) / float(x.shape[0])
        return accuracy

    # x : 입력 데이터, t : 정답 레이블
    def numerical_gradient(self, x, t):
        loss_W = lambda W: self.loss(x, t)

        grads = {}
        grads['W1'] = numerical_gradient(loss_W, self.params['W1'])
        grads['b1'] = numerical_gradient(loss_W, self.params['b1'])
        grads['W2'] = numerical_gradient(loss_W, self.params['W2'])
        grads['b2'] = numerical_gradient(loss_W, self.params['b2'])

        return grads

```

KNU
KANGWON NATIONAL UNIVERSITY

```python
    def gradient(self, x, t):
        W1, W2 = self.params['W1'], self.params['W2']
        b1, b2 = self.params['b1'], self.params['b2']
        grads = {}

        batch_num = x.shape[0]

        # forward
        a1 = np.dot(x, W1) + b1
        z1 = sigmoid(a1)
        a2 = np.dot(z1, W2) + b2
        y = softmax(a2)

        # backward
        dy = (y - t) / batch_num
        grads['W2'] = np.dot(z1.T, dy)
        grads['b2'] = np.sum(dy, axis=0)

        da1 = np.dot(dy, W2.T)
        dz1 = sigmoid_grad(a1) * da1
        grads['W1'] = np.dot(x.T, dz1)
        grads['b1'] = np.sum(dz1, axis=0)

        return grads
```

KNU
KANGWON NATIONAL UNIVERSITY

Thank You!

(Q & A)

References

- 사이토 고키 (2025), 밑바닥부터 시작하는 딥러닝 1(리마스터판), 한빛미디어